

NumPy Guide

Numpy is a powerful library for numerical computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently.

1. Installation & Basics

To install numpy via terminal, you can use:

```
pip install numpy
```

```
In [2]: import numpy as np
```

```
In [3]: # Creating a numpy array
list_vals = [1, 2, 3, 4, 5]
arr = np.array(list_vals)
print("Creating a numpy array:")
print("Array:", arr)
print("Type:", type(arr))

# Creating a numpy array with a specific data type
arr_float = np.array(list_vals, dtype=float)
print("\nCreating a numpy array with a specific data type:")
print("Float Array:", arr_float)
print("Dtype:", arr_float.dtype)

# Creating a numpy matrix
list_2d = [[1, 2, 3], [4, 5, 6]]
matrix = np.array(list_2d)
print("\nCreating a numpy matrix:")
print("Matrix:\n", matrix)
print("Type:", type(matrix))
```

```
Creating a numpy array:
Array: [1 2 3 4 5]
Type: <class 'numpy.ndarray'>
```

```
Creating a numpy array with a specific data type:
Float Array: [1. 2. 3. 4. 5.]
Dtype: float64
```

```
Creating a numpy matrix:
Matrix:
[[1 2 3]
 [4 5 6]]
Type: <class 'numpy.ndarray'>
```

2. Special Arrays and Ranges

```
In [12]: # Zeros Matrix
zeros_array = np.zeros((3, 3))
```

```

print("Zeros Matrix:")
print("Zeros:\n", zeros_array)

# Ones Matrix
ones_array = np.ones((2, 4))
print("\nOnes Matrix:")
print("Ones:\n", ones_array)

# Identity Matrix
eye_array = np.eye(4)
print("\nIdentity Matrix:")
print("Identity:\n", eye_array)

# Full Matrix
full_array = np.full((2, 3), 7)
print("\nFull Matrix:")
print("Full:\n", full_array)

# Ranges
range_array = np.arange(0, 10, 2)
print("\nRanges:")
print("Arange:", range_array)
linspace_array = np.linspace(0, 1, 5)
print("\nLinspace:")
print("Linspace:", linspace_array)

```

Zeros Matrix:

Zeros:

```

[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

```

Ones Matrix:

Ones:

```

[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

```

Identity Matrix:

Identity:

```

[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

```

Full Matrix:

Full:

```

[[7 7 7]
 [7 7 7]]

```

Ranges:

Arange: [0 2 4 6 8]

Linspace:

Linspace: [0. 0.25 0.5 0.75 1.]

3. Array Properties & Random Arrays

```

In [13]: # Array properties
print("Array properties:")

```

```

print("Shape:", matrix.shape)
print("Dimensions:", matrix.ndim)
print("Size:", matrix.size)
print("Dtype:", matrix.dtype)

# Random arrays
print("\nRandom arrays:")

np.random.seed(42) # Set seed for reproducibility
random_array = np.random.rand(3, 3)
print("Random (0 to 1):\n", random_array)

random_int_array = np.random.randint(0, 10, (4, 4))
print("Random Integers:\n", random_int_array)

```

Array properties:

Shape: (2, 3)

Dimensions: 2

Size: 6

Dtype: int64

Random arrays:

Random (0 to 1):

```

[[0.37454012 0.95071431 0.73199394]
 [0.59865848 0.15601864 0.15599452]
 [0.05808361 0.86617615 0.60111501]]

```

Random Integers:

```

[[7 2 5 4]
 [1 7 5 1]
 [4 0 9 5]
 [8 0 9 2]]

```

4. Mathematical, Matrix Operations & Linear Algebra

```

In [15]: a = np.array([1, 2, 3])
         b = np.array([4, 5, 6])

# Element-wise operations
print("Element-wise operations:")
print("Add:", a + b)
print("Sub:", a - b)
print("Mult:", a * b)
print("Div:", a / b)
print("Power:", a ** 2)

# Matrix multiplication/Dot product
print("\nMatrix multiplication/Dot product:")
print("Dot method:", a.dot(b))
print("Np dot:", np.dot(a, b))
print("Operator @:", a @ b)

# Matrix transpose
print("\nMatrix transpose:")
print("Transpose:", a.T)

# Eigenvalues
G = np.array([[1, 7, 2],

```

```

        [5, 4, 3],
        [1, 3, 5]])
eigenvalues = np.linalg.eigvals(G)
print("\nEigenvalues:", eigenvalues)

```

Element-wise operations:

```

Add: [5 7 9]
Sub: [-3 -3 -3]
Mult: [ 4 10 18]
Div: [0.25 0.4 0.5 ]
Power: [1 4 9]

```

Matrix multiplication/Dot product:

```

Dot method: 32
Np dot: 32
Operator @: 32

```

Matrix transpose:

```

Transpose: [1 2 3]

```

```

Eigenvalues: [10.52278388 -3.66244824  3.13966436]

```

5. Indexing, Slicing and Boolean Masking

```

In [16]: arr = np.array([10, 20, 30, 40, 50, 60, 70])

# Indexing
print("Indexing:")
print("First:", arr[0])
print("Last:", arr[-1])

# Slicing
print("\nSlicing:")
print("Slice 1 to 3:", arr[1:4])
print("Slice to 2:", arr[:2])
print("Reverse:", arr[::-1])

# Boolean Masking (Filtering with conditions)
print("\nBoolean Masking:")

mask_and = (arr > 20) & (arr < 60)
print("Mask (20 < x < 60):", arr[mask_and])

mask_or = (arr == 10) | (arr > 50)
print("Mask (x == 10 OR x > 50):", arr[mask_or])

```

Indexing:

```

First: 10
Last: 70

```

Slicing:

```

Slice 1 to 3: [20 30 40]
Slice to 2: [10 20]
Reverse: [70 60 50 40 30 20 10]

```

Boolean Masking:

```

Mask (20 < x < 60): [30 40 50]
Mask (x == 10 OR x > 50): [10 60 70]

```

6. Statistical Functions

```
In [17]: data = np.array([1, 2, 3, 4, 5])

# Basic statistics
print("Basic statistics:")
print("Sum:", np.sum(data))
print("Mean:", np.mean(data))
print("Median:", np.median(data))

# Standard Deviation and Variance (with degrees of freedom)
print("\nStandard Deviation and Variance (with degrees of freedom):")
print("Std Dev:", np.std(data))
print("Std Dev (ddof=1):", np.std(data, ddof=1))
print("Variance:", np.var(data))
print("Variance (ddof=1):", np.var(data, ddof=1))

# Min, Max, Percentiles, Correlation, Covariance
print("\nMin, Max, Percentiles, Correlation, Covariance:")
print("Min:", np.min(data))
print("Max:", np.max(data))
print("25th Percentile:", np.percentile(data, 25))
print("75th Percentile:", np.percentile(data, 75))
print("Correlation Coefficient:\n", np.corrcoef(data, data))
print("Covariance:\n", np.cov(data, data))

# Sorting and Weighted Average
print("\nSorting and Weighted Average:")

unordered_data = np.array([543, 243, 743, 283])
print("Sorted:", np.sort(unordered_data))

weights = np.array([0.1, 0.4, 0.3, 0.2])
print("Weighted Average:", np.average(unordered_data, weights=weights))
```

Basic statistics:

Sum: 15

Mean: 3.0

Median: 3.0

Standard Deviation and Variance (with degrees of freedom):

Std Dev: 1.4142135623730951

Std Dev (ddof=1): 1.5811388300841898

Variance: 2.0

Variance (ddof=1): 2.5

Min, Max, Percentiles, Correlation, Covariance:

Min: 1

Max: 5

25th Percentile: 2.0

75th Percentile: 4.0

Correlation Coefficient:

```
[[1. 1.]
```

```
[1. 1.]]
```

Covariance:

```
[[2.5 2.5]
```

```
[2.5 2.5]]
```

Sorting and Weighted Average:

Sorted: [243 283 543 743]

Weighted Average: 431.0

7. Universal Functions (ufuncs)

```
In [18]: arr = np.array([0, np.pi / 2, np.pi])

# Mathematical functions
print("Mathematical functions:")
print("Sin:", np.sin(arr))
print("Cos:", np.cos(arr))
print("Exp:", np.exp(arr))
print("Log (base e):", np.log(arr + 1))
print("Log10 (base 10):", np.log10(arr + 1))
print("Sqrt:", np.sqrt(arr))
print("Abs:", np.abs(arr - np.pi / 2))
print("Round:", np.round(arr, decimals=2))
print("Floor:", np.floor(arr))
print("Ceil:", np.ceil(arr))
print("Sign:", np.sign(arr - np.pi / 2))
print("Where:", np.where(arr > np.pi / 2, "greater", "not greater"))
print("Unique:", np.unique(arr))
```

Mathematical functions:

```

Sin: [0.0000000e+00 1.0000000e+00 1.2246468e-16]
Cos: [ 1.0000000e+00  6.123234e-17 -1.0000000e+00]
Exp: [ 1.          4.81047738 23.14069263]
Log (base e): [0.          0.94421571 1.42108041]
Log10 (base 10): [0.          0.41006767 0.61716738]
Sqrt: [0.          1.25331414 1.77245385]
Abs: [1.57079633 0.          1.57079633]
Round: [0.    1.57 3.14]
Floor: [0. 1. 3.]
Ceil: [0. 2. 4.]
Sign: [-1.  0.  1.]
Where: ['not greater' 'not greater' 'greater']
Unique: [0.          1.57079633 3.14159265]

```

8. Write and Read from Files

```

In [23]: arr = np.array([[1, 2, 3], [4, 5, 6]])

# Basic TXT Saving/Loading
np.savetxt("./assets/gen_array.txt", arr, delimiter=",")
loaded_arr = np.loadtxt("./assets/gen_array.txt", delimiter=",")
print("Basic TXT Saving/Loading:")
print("Loaded TXT:\n", loaded_arr)

# NPY Saving/Loading (Binary, much faster)
np.save("./assets/gen_array.npy", arr)
loaded_arr_npy = np.load("./assets/gen_array.npy")
print("\nNPY Saving/Loading (Binary, much faster):")
print("Loaded NPY:\n", loaded_arr_npy)

# Advanced TXT Loading (np.genfromtxt)
# Allows skipping specific lines (e.g., text headers/footers) and selecti
data_from_txt = np.genfromtxt('./assets/read_array.txt', skip_header=1, s
print("\nAdvanced TXT Loading (np.genfromtxt):")
print("Loaded with genfromtxt:\n", data_from_txt)

```

Basic TXT Saving/Loading:

```

Loaded TXT:
[[1. 2. 3.]
 [4. 5. 6.]]

```

NPY Saving/Loading (Binary, much faster):

```

Loaded NPY:
[[1 2 3]
 [4 5 6]]

```

Advanced TXT Loading (np.genfromtxt):

```

Loaded with genfromtxt:
[[ 1.  3.]
 [ 4.  6.]
 [ 7.  9.]
[10. 12.]]

```